

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

Факултет по компютърни системи и технологии

Специалност „Компютърно и софтуерно инженерство“

К У Р С О В А Р А Б О Т А

(реферат) по дисциплината „Операционни системи“

на тема:

Планиране на процесорното време

сравнение на FCFS, SJF, Round Robin и приоритетно планиране

Изготвил:
Алекс Александров Огнянов фак. № 121224052
КСИ, III курс група 43

Проверил:
……
/ Боян Петров /

София, 2026/2027 учебна година, зимен семестър

СЪДЪРЖАНИЕ

Увод 3
1. Основни понятия 5
1.1. Процеси, нишки и техните състояния 5
1.2. Редуване на CPU burst и I/O burst 6
1.3. Планировчик, диспечер и видове планиране 6
1.4. Критерии за оценка на алгоритмите 7
1.5. Пример, използван в реферата 8
2. Алгоритъм FCFS 10
3. Алгоритми SJF и SRTF 12
4. Алгоритъм Round Robin 15
5. Приоритетно планиране 17
6. Сравнителен анализ 20
6.1. Методика 20
6.2. Резултати за общия пример 20
6.3. Смесено натоварване от 200 процеса 21
6.4. Влияние на кванта при Round Robin 22
6.5. Обобщено сравнение 24
7. Планиране в съвременните операционни системи 26
7.1. Многостепенни опашки с обратна връзка 26
7.2. Linux 27
7.3. Windows 28
Заключение 30
Използвана литература 32
Приложение А. Програма за симулиране на алгоритмите 33

УВОД

В една съвременна компютърна система едновременно съществуват десетки или стотици процеси, а процесорните ядра са малко. Във всеки момент операционната система трябва да реши кой от готовите за изпълнение процеси да получи процесора и за колко време. Тази задача се нарича **планиране на процесорното време** (CPU scheduling), а частта от ядрото, която я решава – **планировчик** (scheduler).

Планирането е в основата на многозадачността. От него зависи дали системата ще използва процесора пълноценно, колко бързо ще завършват пакетните задачи и дали интерактивните програми ще реагират без забавяне на действията на потребителя. Тези цели често си противоречат: алгоритъм, който минимизира средното време за изпълнение, може да остави дълъг процес да чака неопределено дълго, а алгоритъм, който дава бърз отговор на всички, губи време за честото превключване между процесите. Затова няма един „най-добър“ алгоритъм, а избор между компромиси според предназначението на системата.

Целта на настоящата курсова работа е да представи и сравни четирите класически алгоритъма за планиране на процесорното време – FCFS (First-Come, First-Served), SJF (Shortest Job First) заедно с варианта му с изместване SRTF, Round Robin и приоритетното планиране – и да покаже как се отразяват на производителността на системата.

За постигане на целта са поставени следните **задачи**:

- да се въведат основните понятия: състояния на процеса, планировчик и диспечер, планиране с и без изместване, критерии за оценка;
- да се опише принципът на работа на всеки алгоритъм с пример, диаграма на Гант и изчислени времена за чакане, обработка и отговор;
- да се сравнят алгоритмите количествено – върху общ пример и върху симулирано смесено натоварване от 200 процеса – и да се изследва влиянието на кванта при Round Robin;
- да се покаже как класическите алгоритми са съчетани в планировчиците на Linux и Windows.

Обхват. Разглежда се краткосрочното планиране на процеси (нишки) на едно процесорно ядро. Многопроцесорното планиране, балансирането на натоварването

между ядрата и строгите системи за реално време са споменати само дотолкова, доколкото са нужни за описанието на Linux и Windows. Описанието на Linux се отнася до ядрата от серия 6.x, а на Windows – до Windows 10 и 11.

Подход. Теоретичната част се основава на класическите учебници по операционни системи [1–4] и на официалната документация на Linux и Windows [5–8, 11]. За сравнението е написана малка програма на Python (Приложение А), която симулира всички алгоритми. Така всички диаграми и числа в работата са получени от един и същ код и могат да бъдат проверени и повторени.

Работата е структурирана в седем раздела. Раздел 1 въвежда понятията и критериите. Раздели 2–5 описват поотделно FCFS, SJF/SRTF, Round Robin и приоритетното планиране. Раздел 6 съдържа сравнителния анализ, а раздел 7 – приложението на алгоритмите в реални операционни системи.

1. ОСНОВНИ ПОНЯТИЯ

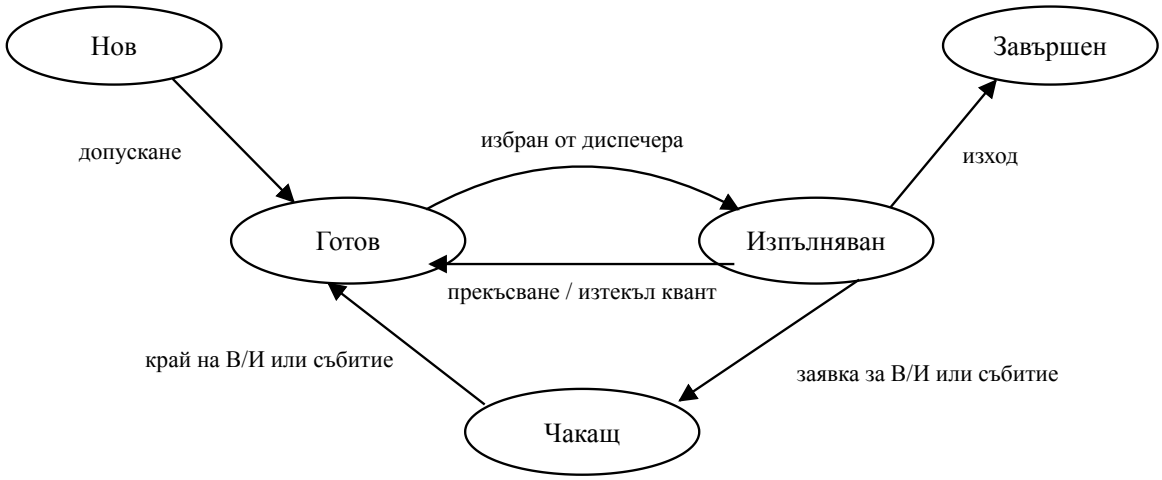
1.1. Процеси, нишки и техните състояния

Процесът е програма в процес на изпълнение: кодът ѝ заедно с текущото състояние – стойностите на регистрите, програмния брояч, стека, адресното пространство и отворените ресурси. Операционната система пази тази информация в структура, наречена **блок за управление на процеса** (PCB, Process Control Block). В Linux тази структура е `task_struct`, а в Windows – `EPROCESS` и `ETHREAD`.

Един процес може да съдържа няколко **нишки** (threads) – независими потоци на изпълнение, които споделят адресното пространство на процеса, но имат собствени регистри и стек. В съвременните операционни системи единицата, която планировчикът разпределя по процесорите, е именно нишката. За простота в реферата се използва думата „процес“, но всичко казано важи и за нишките.

По време на своето съществуване процесът преминава през няколко състояния (фиг. 1):

- **нов** – процесът се създава;
- **готов** (ready) – процесът има всичко необходимо и чака само процесора;
- **изпълняван** (running) – инструкциите му се изпълняват от процесора;
- **чакащ** или блокиран (waiting) – процесът чака събитие, най-често завършване на входно-изходна операция;
- **завършен** – изпълнението е приключило и ресурсите се освобождават.



Фиг. 1. Диаграма на състоянията на процеса

Готовите процеси се подреждат в **опашка на готовите процеси** (ready queue). Планирането на процесорното време по същество е правилото, по което от тази опашка се избира следващият процес, и правилото, по което изпълняваният процес се връща в нея.

1.2. Редуване на CPU burst и I/O burst

Наблюденията върху реални програми показват, че изпълнението на процеса се състои от редуващи се интервали: процесорна работа (**CPU burst**), последвана от чакане на вход-изход (**I/O burst**), след това отново CPU burst и т.н. Последният CPU burst завършва със системно извикване за край на процеса.

Продължителността на CPU burst-овете има характерно разпределение: голям брой кратки burst-ове и малък брой дълги [1]. Според това процесите се делят на две групи:

- **процеси, ограничени от входа-изхода** (I/O-bound) – текстови редактори, браузъри, сървъри; имат много кратки CPU burst-ове и често се блокират;
- **процеси, ограничени от процесора** (CPU-bound) – компилация, научни изчисления, кодиране на видео; имат малко, но дълги CPU burst-ове.

Добрият планировчик трябва да обслужва бързо първата група, за да бъдат входно-изходните устройства натоварени и системата да реагира бързо, без да оставя без процесорно време втората група. Именно продължителността на следващия CPU burst е величината, на която се основават алгоритмите SJF и SRTF.

1.3. Планировчик, диспечер и видове планиране

Според това колко често се извършва и какво решава, планирането се дели на три нива [3]:

- **дългосрочно планиране** – кои нови задачи да бъдат допуснати в системата; определя степента на многозадачност и е характерно за пакетните системи;
- **средносрочно планиране** – временно изваждане на процеси от паметта (swapping) и връщането им обратно при недостиг на памет;
- **краткосрочно планиране** – кой от готовите процеси да получи процесора сега. То се извършва много често (стотици до хиляди пъти в секунда) и

затова трябва да е бързо. Темата на реферата е именно краткосрочното планиране.

Изборът на процес се прави от **планировчика**, а самото предаване на процесора – от **диспечера** (dispatcher). Диспечерът превключва контекста (запазва регистрите на стария процес в неговия PCB и зарежда тези на новия), превключва в потребителски режим и прехвърля управлението на подходящото място в новия процес. Времето за това е **закъснение на диспечера** (dispatch latency) и представлява чиста загуба – през него не се върши полезна работа. В съвременните системи едно превключване на контекста отнема от порядъка на микросекунди, но косвената цена от изчистените кешове и TLB може да е значително по-голяма.

Решение за планиране може да се наложи в четири ситуации [1]:

1. изпълняваният процес преминава в състояние „чакащ“ (например заявка за вход-изход);
2. изпълняваният процес преминава в състояние „готов“ (например при прекъсване от таймера);
3. чакащ процес става готов (например завършва входно-изходна операция);
4. изпълняваният процес завършва.

Ако планировчикът се извиква само в случаи 1 и 4, планирането е **без изместване** (non-preemptive, кооперативно): веднъж получил процесора, процесът го държи, докато сам не го освободи. Ако планировчикът може да отнеме процесора и в случаи 2 и 3, планирането е **с изместване** (preemptive). Всички съвременни операционни системи с общо предназначение – Windows, macOS, Linux – използват планиране с изместване. То позволява бърза реакция и защитава системата от процес, който не освобождава процесора, но изисква синхронизация при достъп до споделени данни, защото процесът може да бъде прекъснат по всяко време.

1.4. Критерии за оценка на алгоритмите

За сравнение на алгоритмите се използват критериите в табл. 1. Първите два се стремим да максимизираме, останалите – да минимизираме.

Таблица 1. Критерии за оценка на алгоритмите за планиране

Критерий	**Определение**
Натоварване на процесора (CPU utilization)	Частта от времето, през която процесорът върши полезна работа. В реални системи е между 40 % (слабо натоварена) и 90 % (силно натоварена).
Пропускателна способност (throughput)	Брой завършени процеси за единица време.
Време за обработка (turnaround time)	Времето от пристигането на процеса до неговото завършване: $T_{обр} = T_{край} - T_{пристигане}$.
Време за чакане (waiting time)	Сумарното време, което процесът прекарва в опашката на готовите процеси: $T_{чак} = T_{обр} - CPU\ burst$. Алгоритъмът за планиране влияе пряко само на тази величина.
Време за отговор (response time)	Времето от пристигането до първото получаване на процесора. Важно е за интерактивните системи.
Справедливост (fairness)	Дали всеки процес получава разумен дял от процесора и дали има процеси, които чакат неограничено дълго (гладуване, starvation).

Обикновено се оптимизира средната стойност на даден критерий. За интерактивните системи обаче често е по-важно да се намали **разсейването** и максималната стойност на времето за отговор: потребителят предпочита система, която винаги отговаря за 0,2 s, пред такава, която средно отговаря за 0,1 s, но понякога не реагира по няколко секунди.

1.5. Пример, използван в реферата

За да могат алгоритмите да се сравнят пряко, в раздели 2–5 всички те се прилагат върху един и същ набор от пет процеса (табл. 2). Времето е в условни единици (например милисекунди), а по-малкото число означава по-висок приоритет. Приема се, че всеки процес има един CPU burst и не извършва вход-изход, а времето за превключване на контекста се пренебрегва.

Таблица 2. Входни данни на общия пример

Процес	**Момент на пристигане**	**CPU burst**	**Приоритет**
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2
P5	4	2	5

Резултатите за всеки алгоритъм са представени с **диаграма на Гант** – времева ос, върху която е отбелязано кой процес се изпълнява във всеки интервал – и с таблица на времената за всеки процес. Диаграмите и таблиците са генерирани от програмата в Приложение А.

2. АЛГОРИТЪМ FCFS

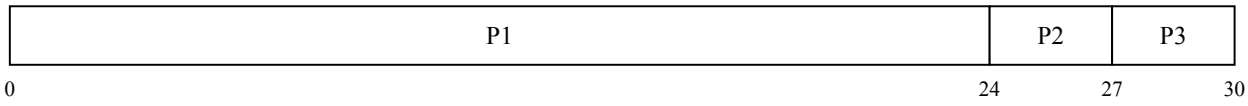
2.1. Принцип на работа

FCFS (First-Come, First-Served – „пръв дошъл, пръв обслужен“) е най-простият алгоритъм за планиране. Процесорът се дава на процесите в реда на пристигането им в опашката на готовите процеси. Опашката е обикновена FIFO опашка: новият процес се добавя в края ѝ, а когато процесорът се освободи, се избира процесът от началото ѝ. FCFS е алгоритъм **без изместване** – процесът се изпълнява, докато не завърши или не се блокира.

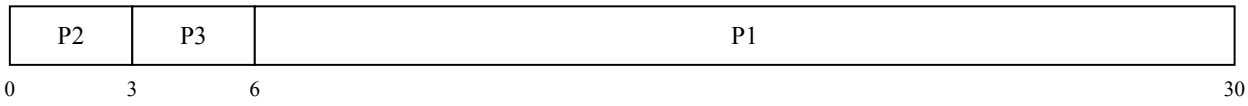
Реализацията е тривиална: добавянето и изваждането от опашката отнемат O(1) време, а планировчикът не се нуждае от никаква информация за процесите.

2.2. Ефект на конвоя

Основният недостатък на FCFS е, че средното време за чакане силно зависи от реда на пристигане. Нека три процеса пристигат едновременно с CPU burst-ове 24, 3 и 3 единици. Ако първи е дългият процес P1 (фиг. 2), кратките процеси P2 и P3 чакат 24 и 27 единици, а средното време за чакане е (0 + 24 + 27) / 3 = 17. Ако кратките процеси са първи (фиг. 3), средното време за чакане е (0 + 3 + 6) / 3 = 3 – пет пъти по-малко при същото натоварване.



Фиг. 2. FCFS, дългият процес пристига първи: средно време за чакане 17

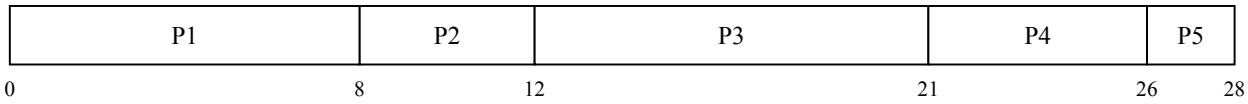


Фиг. 3. FCFS, кратките процеси пристигат първи: средно време за чакане 3

Това явление се нарича **ефект на конвоя** (convoy effect): много кратки процеси се „натрупват“ зад един дълъг, както леки коли зад бавен камион. В реална система с един CPU-bound и много I/O-bound процеси резултатът е още по-лош: докато CPU-bound процесът държи процесора, I/O-bound процесите завършват своя вход-изход и чакат, а входно-изходните устройства бездействат. След това всички I/O-bound процеси бързо преминават през процесора и отново се блокират, а сега бездейства процесорът.

2.3. Резултати за общия пример

При данните от табл. 2 процесите се изпълняват в реда P1, P2, P3, P4, P5 (фиг. 4). Най-краткият процес P5 пристига последен и чака 22 единици, за да се изпълни за 2.



Фиг. 4. Диаграма на Гант при FCFS

Таблица 3. Времена при FCFS

Процес	Пристигане	Burst	Край	Обработка	Чакане	Отговор
P1	0	8	8	8	0	0
P2	1	4	12	11	7	7
P3	2	9	21	19	10	10
P4	3	5	26	23	18	18
P5	4	2	28	24	22	22
Средно				**17,00**	**11,40**	**11,40**

2.4. Предимства и недостатъци

- **Предимства:** изключително проста реализация; минимални разходи за планиране и малко превключвания на контекста; справедлив в смисъл, че няма гладуване – всеки процес рано или късно получава процесора.
- **Недостатъци:** ефект на конвоя и високо средно време за чакане; лошо време за отговор; неподходящ за интерактивни системи и системи с разделяне на времето, защото един процес може да заеме процесора за дълго.

Самостоятелно FCFS се използва рядко, но е важен градивен елемент: процеси с еднакъв приоритет в много планировчици се обслужват по реда на пристигане, а политиката `SCHED_FIFO` в Linux е FCFS в рамките на едно ниво на приоритет (раздел 7.2).

3. АЛГОРИТМИ SJF И SRTF

3.1. Принцип на работа

SJF (Shortest Job First – „първо най-кратката задача“, наричан още Shortest Job Next) избира от опашката на готовите процеси този с най-кратък следващ CPU burst. При равенство се прилага FCFS. Класическият SJF е без изместване: избраният процес се изпълнява докрай.

Вариантът **с изместване** се нарича **SRTF** (Shortest Remaining Time First – „първо най-малкото оставащо време“). Когато в опашката пристигне нов процес, планировчикът сравнява неговия CPU burst с *оставащото* време на изпълнявания процес. Ако новият е по-кратък, изпълняваният процес се измества.

3.2. Оптималност

SJF има важно теоретично свойство: **за даден набор от процеси, пристигнали едновременно, SJF дава минималното възможно средно време за чакане** [1, 2, 4]. Доказателството е просто: ако кратък процес се премести преди дълъг, времето за чакане на краткия намалява с дължината на дългия, а това на дългия се увеличава само с дължината на краткия, т.е. общата сума намалява. Следователно всяка подредба, различна от възходящата по дължина, може да бъде подобрена.

Когато процесите пристигат по различно време, оптимален сред алгоритмите с изместване е SRTF. Затова SJF/SRTF служат като еталон: никой реален алгоритъм не може да постигне по-малко средно време за чакане.

3.3. Предсказване на дължината на CPU burst

Голямата практическа трудност е, че планировчикът **не знае** колко ще продължи следващият CPU burst. При дългосрочното планиране в пакетните системи потребителят може да посочи ограничение на времето, но при краткосрочното планиране това е невъзможно. Затова дължината се предсказва по историята на процеса чрез **експоненциално усредняване**:

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n, \quad 0 \le \alpha \le 1,$$

където t_n е действителната дължина на n-тия CPU burst, τ_n е предишната прогноза, а τ_{n+1} е прогнозата за следващия burst. Параметърът α определя тежестта на последното наблюдение: при α = 0 историята се пренебрегва, при α = 1 се взема само последният burst. Обикновено α = 1/2. След разгръщане се вижда, че по-старите наблюдения влизат с тегла $(1 - \alpha)^k$, т.е. влиянието им намалява експоненциално.

Например при α = 1/2, начална прогноза $\tau_0 = 10$ и действителни burst-ове 6, 4, 6, 4 прогнозите са 8, 6, 6, 5 – прогнозата бързо се приближава към реалното поведение на процеса.

3.4. Резултати за общия пример

При **SJF** (фиг. 5) в момент 0 е готов само P1 и той се изпълнява докрай, въпреки че след момент 1 пристигат по-кратки процеси. В момент 8 се избират последователно най-кратките: P5 (2), P2 (4), P4 (5) и накрая P3 (9).

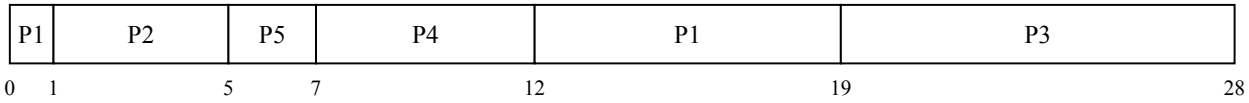


Фиг. 5. Диаграма на Гант при SJF (без изместване)

Таблица 4. Времена при SJF

Процес	Пристигане	Burst	Край	Обработка	Чакане	Отговор
P1	0	8	8	8	0	0
P2	1	4	14	13	9	9
P3	2	9	28	26	17	17
P4	3	5	19	16	11	11
P5	4	2	10	6	4	4
Средно				**13,80**	**8,20**	**8,20**

При **SRTF** (фиг. 6) в момент 1 пристига P2 с burst 4, което е по-малко от оставащите 7 единици на P1, и P1 се измества. В момент 4 пристига P5 с burst 2, но на P2 му остава само 1 единица, така че P2 не се измества. След това се изпълняват P5, P4, остатъкът от P1 и накрая P3. Средното време за чакане е най-малкото сред всички разгледани алгоритми.



Фиг. 6. Диаграма на Гант при SRTF (SJF с изместване)

Таблица 5. Времена при SRTF

Процес	Пристигане	Burst	Край	Обработка	Чакане	Отговор
P1	0	8	19	19	11	0
P2	1	4	5	4	0	0
P3	2	9	28	26	17	17
P4	3	5	12	9	4	4
P5	4	2	7	3	1	1
Средно				**12,20**	**6,60**	**4,40**

3.5. Предимства и недостатъци

- **Предимства:** минимално средно време за чакане и обработка; висока пропускателна способност; кратките интерактивни процеси се обслужват бързо.
- **Недостатъци:** дължината на CPU burst не е известна и трябва да се предсказва; възможно е **гладуване** – при постоянен поток от кратки процеси дългият процес може никога да не получи процесора; SRTF изисква сравнение при всяко пристигане и води до повече превключвания; алгоритъмът не отчита важността на процесите.

Чист SJF почти не се използва в операционните системи с общо предназначение, но идеята му – да се дава предимство на процесите с кратки CPU burst-ове – присъства неявно в многостепенните опашки с обратна връзка (раздел 7.1) и в планировчиците на Linux и Windows.

4. АЛГОРИТЪМ ROUND ROBIN

4.1. Принцип на работа

Round Robin (RR, „циклично обслужване“) е проектиран специално за системите с разделяне на времето. Той е FCFS с изместване по таймер: определя се малка единица време, наречена **квант** (time quantum, time slice), обикновено от 10 до 100 ms. Опашката на готовите процеси е кръгова FIFO опашка. Планировчикът взема първия процес, настройва таймера да генерира прекъсване след един квант и му дава процесора. Възможни са два случая:

- CPU burst-ът на процеса е по-кратък от кванта – процесът сам освобождава процесора и планировчикът избира следващия;
- CPU burst-ът е по-дълъг – таймерът прекъсва процеса, контекстът му се запазва и той се поставя в края на опашката.

Ако в опашката има n процеса и квантът е q, всеки процес получава $1/n$ от процесорното време на порции от най-много q единици и **никога не чака повече от $(n-1)\cdot q$** единици до следващия си квант. Тази гаранция прави RR подходящ за интерактивни системи.

4.2. Избор на кванта

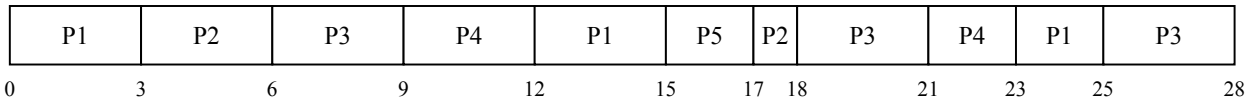
Производителността на RR силно зависи от размера на кванта:

- при **много голям квант** (по-голям от всички CPU burst-ове) RR се превръща във FCFS и губи предимството си за времето за отговор;
- при **много малък квант** процесите сякаш се изпълняват едновременно, всеки на процесор с $1/n$ от скоростта (processor sharing), но голяма част от времето отива за превключване на контекста. Ако превключването отнема 1 единица, а квантът е 4, около 20 % от процесорното време се губи.

Практическото правило е квантът да е значително по-голям от времето за превключване на контекста и около 80 % от CPU burst-овете да са по-кратки от него [1]. Така повечето процеси завършват burst-а си в рамките на един квант и превключване поради изтичане на кванта има рядко. Влиянието на кванта е изследвано количествено в раздел 6.4.

4.3. Резултати за общия пример

За общия пример е използван квант $q = 3$ (фиг. 7). При пристигане на процес в същия момент, в който изтича квантът на друг, новопристигналият се поставя в опашката преди изместения процес – това е обичайната конвенция и в учебниците [1].



Фиг. 7. Диаграма на Гант при Round Robin с квант 3

Таблица 6. Времена при Round Robin, q = 3

Процес	Пристигане	Burst	Край	Обработка	Чакане	Отговор
P1	0	8	25	25	17	0
P2	1	4	18	17	13	2
P3	2	9	28	26	17	4
P4	3	5	23	20	15	6
P5	4	2	17	13	11	11
Средно				**20,20**	**14,60**	**4,60**

Резултатът показва типичната картина за RR: средното време за отговор (4,6) е почти колкото при SRTF и много по-добро от FCFS, но средното време за обработка е най-лошото, защото всеки процес се изпълнява на части и завършва късно. Броят превключвания на контекста е 10 – два пъти повече от останалите алгоритми.

4.4. Предимства и недостатъци

- **Предимства:** справедлив – всички процеси получават равен дял; няма гладуване; гарантирано горно ограничение на времето до следващото изпълнение; не изисква знание за дължината на CPU burst; добро време за отговор.
- **Недостатъци:** по-голямо средно време за обработка, особено когато процесите са с еднаква дължина (всички завършват почти накрая); разходи за чести превключвания на контекста; не отчита важността на процесите; I/O-bound процесите, които използват само малка част от кванта си, са в неизгодно положение спрямо CPU-bound процесите.

5. ПРИОРИТЕТНО ПЛАНИРАНЕ

5.1. Принцип на работа

При **приоритетното планиране** на всеки процес се присвоява число – **приоритет**, и процесорът се дава на готовия процес с най-висок приоритет. Процесите с еднакъв приоритет се обслужват по FCFS или по Round Robin. Няма общо съгласие дали малкото число означава висок или нисък приоритет: в Linux и UNIX (стойността *nice*) по-малкото число е по-висок приоритет, а в Windows – обратно. В реферата по-малкото число е по-висок приоритет.

SJF е частен случай на приоритетното планиране, при който приоритетът е обратно пропорционален на предсказаната дължина на CPU burst. Приоритетите могат да се определят:

- **вътрешно** – от операционната система по измерими величини: изисквания към паметта, брой отворени файлове, отношение между средния I/O burst и средния CPU burst;
- **външно** – по критерии извън операционната система: важност на задачата, вид на потребителя, платена услуга.

Приоритетите са **статични**, ако не се променят по време на изпълнението, и **динамични**, ако операционната система ги коригира според поведението на процеса. Приоритетното планиране може да бъде **без изместване** (новият процес с висок приоритет се поставя в началото на опашката, но изчаква изпълнявания) и **с изместване** (новият процес веднага отнема процесора от процес с по-нисък приоритет).

5.2. Гладуване и стареене

Основният проблем на приоритетното планиране е **гладуването** (starvation, indefinite blocking): при постоянен поток от процеси с висок приоритет процес с нисък приоритет може никога да не получи процесора. Известен е разказът, че при спирането на компютъра IBM 7094 в MIT през 1973 г. е открит процес с нисък приоритет, подаден през 1967 г. и все още неизпълнен [1].

Решението е **стареенето** (aging): приоритетът на процес, който дълго чака в опашката, постепенно се повишава. Например приоритетът може да се повишава с единица на всеки 50 единици чакане – така дори процес с най-нисък приоритет

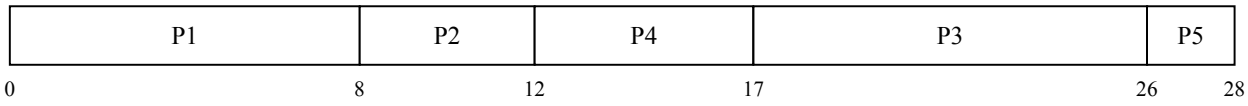
след определено време достига най-високия и бива изпълнен. В симулацията в раздел 6.3 е използван точно този вариант.

5.3. Инверсия на приоритетите

При приоритетното планиране с изместване и споделени ресурси може да възникне **инверсия на приоритетите** (priority inversion): процес с висок приоритет H чака ресурс (например мютекс), заключен от процес с нисък приоритет L, а процес със среден приоритет M измества L и така косвено забавя H. Известен случай е с марсианския апарат Mars Pathfinder през 1997 г., при който инверсия на приоритетите в операционната система за реално време VxWorks водела до периодични рестартирания [1]. Стандартното решение е **наследяването на приоритета** (priority inheritance): докато L държи ресурса, нужен на H, той временно получава приоритета на H.

5.4. Резултати за общия пример

При приоритетното планиране **без изместване** (фиг. 8) P1 е единственият готов процес в момент 0 и се изпълнява докрай. След това процесите се изпълняват по приоритет: P2 (1), P4 (2), P3 (4), P5 (5). Процесът P5 е най-кратък, но с най-нисък приоритет, затова завършва последен.

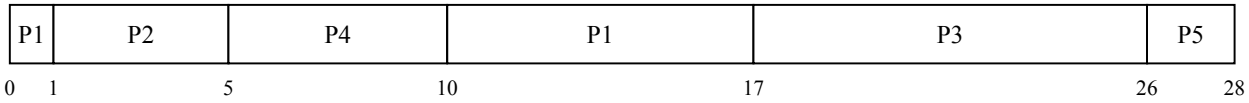


Фиг. 8. Диаграма на Гант при приоритетно планиране без изместване

Таблица 7. Времена при приоритетно планиране без изместване

Процес	Пристигане	Burst	Край	Обработка	Чакане	Отговор
P1	0	8	8	8	0	0
P2	1	4	12	11	7	7
P3	2	9	26	24	15	15
P4	3	5	17	14	9	9
P5	4	2	28	24	22	22
Средно				**16,20**	**10,60**	**10,60**

При планиране **с изместване** (фиг. 9) P2 с приоритет 1 отнема процесора от P1 (приоритет 3) веднага при пристигането си в момент 1. В момент 5 P4 (приоритет 2) е с по-висок приоритет от P1 и се изпълнява преди остатъка от него.



Фиг. 9. Диаграма на Гант при приоритетно планиране с изместване

Таблица 8. Времена при приоритетно планиране с изместване

Процес	Пристигане	Burst	Край	Обработка	Чакане	Отговор
P1	0	8	17	17	9	0
P2	1	4	5	4	0	0
P3	2	9	26	24	15	15
P4	3	5	10	7	2	2
P5	4	2	28	24	22	22
Средно				**15,20**	**9,60**	**7,80**

5.5. Предимства и недостатъци

- **Предимства:** отчита важността на процесите; гъвкав – с подходящ избор на приоритетите може да имитира FCFS, SJF и други политики; позволява бърза реакция на критичните процеси, което е задължително в системите за реално време.
- **Недостатъци:** гладуване на процесите с нисък приоритет, ако няма стареене; инверсия на приоритетите при споделени ресурси; ефективността зависи от това доколко приоритетите отразяват реалното поведение на процесите.

6. СРАВНИТЕЛЕН АНАЛИЗ

6.1. Методика

За количественото сравнение е написана програма на Python 3 (Приложение А), която симулира шестте варианта на алгоритмите: FCFS, SJF, SRTF, Round Robin и приоритетно планиране без и с изместване, а за раздел 6.3 – и приоритетно планиране със стареене. Времето в симулатора е дискретно: на всяка стъпка програмата добавя новопристигналите процеси в опашката, проверява дали изпълняваният процес трябва да бъде изместен, при нужда избира нов процес и изпълнява една единица от него. Резултатът е диаграма на Гант и времената за всеки процес. По желание симулаторът добавя цена на превключването на контекста – определен брой единици, през които процесорът не върши полезна работа.

Сравнението е направено в три стъпки: общият пример от табл. 2; синтетично смесено натоварване от 200 процеса; Round Robin с различен размер на кванта при отчитане на цената на превключването.

6.2. Резултати за общия пример

Табл. 9 обобщава средните времена от табл. 3–8.

Таблица 9. Сравнение на алгоритмите за общия пример

Алгоритъм	Ср. време за обработка	Ср. време за чакане	Ср. време за отговор	Превключвания
FCFS	17,00	11,40	11,40	4
SJF	13,80	8,20	8,20	4
SRTF	12,20	6,60	4,40	5
Round Robin (q = 3)	20,20	14,60	4,60	10
Приоритетно (без изместване)	16,20	10,60	10,60	4
Приоритетно (с изместване)	15,20	9,60	7,80	5

Резултатите потвърждават теорията:

- **SRTF** дава най-малкото средно време за чакане (6,6) и обработка (12,2), а SJF без изместване е на второ място. И двата алгоритъма поставят дългия

P3 последен.

- **FCFS** има почти два пъти по-голямо средно време за чакане от SRTF, защото P4 и P5 изчакват дългите P1 и P3 (ефект на конвоя).
- **Round Robin** има най-голямо средно време за обработка, но второто най-добро средно време за отговор – всеки процес получава процесора в рамките на няколко единици след пристигането си.
- **Приоритетните алгоритми** дават средни резултати, защото приоритетите в примера не са свързани с дължината на процесите. За сметка на това P2, процесът с най-висок приоритет, е обслужен веднага при планирането с изместване.

6.3. Смесено натоварване от 200 процеса

Пет процеса са твърде малко за общи изводи. Затова симулаторът генерира натоварване, близко до реалното: 200 процеса с експоненциално разпределени интервали между пристиганията (средно 38 единици); 80 % от процесите са кратки, интерактивни (CPU burst от 2 до 20 единици), а 20 % са дълги, изчислителни (от 50 до 200 единици); приоритетите са случайни от 1 до 8. Общото процесорно време е 6828 единици, а последният процес пристига в момент 7716, т.е. натоварването на процесора е около 88 %. Генераторът използва фиксирано начално число, така че резултатите са повторими.

Таблица 10. Сравнение при смесено натоварване (200 процеса, без цена на превключването)

Алгоритъм	Ср. обработка	Ср. чакане	Ср. отговор	Макс. чакане	Превключвания
FCFS	232,5	198,4	198,4	547	199
SJF	119,7	85,5	85,5	1547	199
SRTF	73,2	39,1	27,9	1604	312
Round Robin (q = 4)	145,0	110,9	13,9	913	1491
Round Robin (q = 20)	149,1	114,9	60,5	837	385
Приоритетно (без изместване)	238,7	204,5	204,5	1747	199
Приоритетно (с изместване)	236,1	202,0	171,1	1747	266
Приоритетно със стареене	247,2	213,0	213,0	724	199

От табл. 10 се виждат следните закономерности:

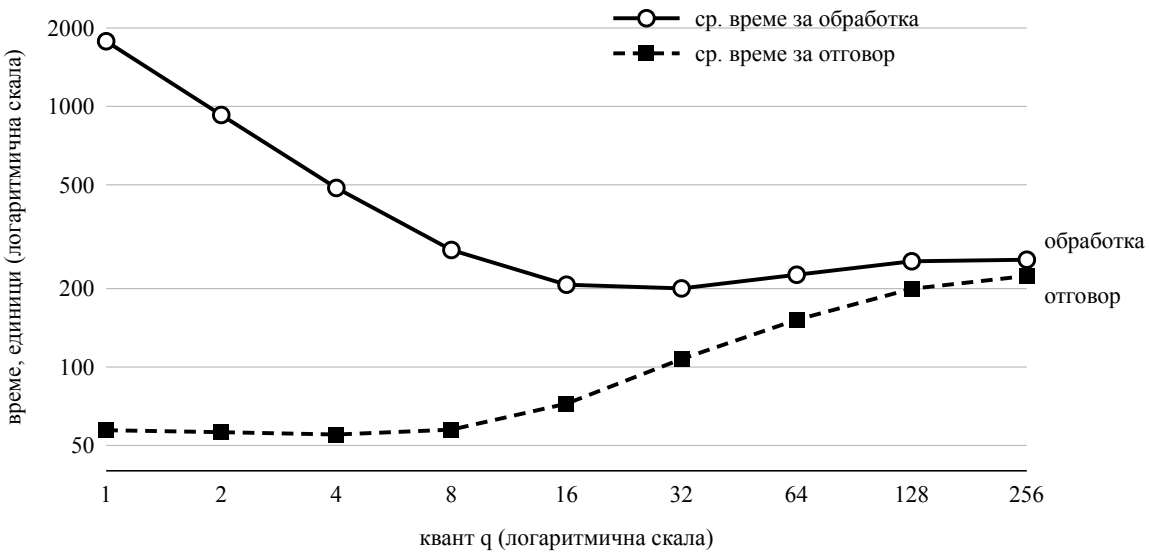
- **SRTF** намалява средното време за чакане около пет пъти спрямо FCFS, а SJF – около два пъти. Цената е в максималното време за чакане: при SJF и SRTF един дълъг процес чака над 1500 единици, тъй като кратките процеси постоянно го изпреварват. Това е гладуването на дългите процеси в числа.
- **FCFS** има най-малкото максимално време за чакане (547) – той е лош средно, но предвидим и без гладуване.
- **Round Robin** с квант 4 има несравнимо най-добро средно време за отговор (около 14 единици срещу около 200 при FCFS), като максималното време за чакане остава умерено. С квант 20 времето за отговор се увеличава над четири пъти, а превключванията намаляват около четири пъти – компромисът, разгледан в раздел 6.4.
- **Приоритетното планиране** със случайни приоритети не подобрява средните стойности спрямо FCFS, но има най-лошото максимално време за чакане (1747): процесите с приоритет 8 гладуват. **Стареенето** намалява максималното време за чакане повече от два пъти (до 724) при почти същите средни стойности.

6.4. Влияние на кванта при Round Robin

За да се оцени влиянието на кванта, същото натоварване е симулирано с Round Robin при кванти от 1 до 256 единици, като всяко превключване на контекста струва 1 единица. Колоната „загуба“ показва каква част от общото време процесорът е прекарал в превключване.

Таблица 11. Round Robin при различен квант (цена на превключването 1 единица)

Квант q	Ср. време за обработка	Ср. време за отговор	Превключвания	Загуба за превключване, %
1	1777,5	57,2	6569	48,22
2	927,2	56,3	3324	31,78
4	486,5	55,1	1714	19,29
8	281,1	57,5	915	11,29
16	206,9	72,4	494	6,26
32	200,2	107,3	319	4,07
64	226,1	151,9	257	3,29
128	254,4	199,6	222	2,85
256	257,9	223,8	199	2,56



Фиг. 10. Средно време за обработка и за отговор при Round Robin в зависимост от кванта

Резултатите показват ясно двата края на компромиса:

- При **квант 1** почти половината процесорно време (48 %) отива за превключване. Полезната работа вече не може да се справи с натоварването от 88 %, опашката расте и средното време за обработка надхвърля 1700 единици – системата е претоварена от собствените си превключвания. Дори времето за отговор не е по-добро, отколкото при квант 4.
- При **много голям квант** (128–256) превключванията стават толкова, колкото при FCFS, а и средното време за отговор се доближава до това при

FCFS.

- Най-доброто средно време за обработка е при **квант 32**, а между 8 и 32 загубата е под 12 %, а времето за отговор остава приемливо. Повечето кратки процеси в натоварването (2–20 единици) завършват в рамките на един квант от 16–32 единици, което съвпада с правилото за 80 % от раздел 4.2.

В реалните системи отношението е много по-благоприятно, отколкото в симулацията: превключването на контекста отнема микросекунди, а квантът е десетки милисекунди, т.е. загубата е под 1 %. Въпреки това зависимостта е същата и обяснява защо квантът не се прави произволно малък.

6.5. Обобщено сравнение

Таблица 12. Качествено сравнение на алгоритмите

Характеристика	**FCFS**	**SJF / SRTF**	**Round Robin**	**Приоритетно**
Изместване	не	SJF – не, SRTF – да	да (по таймер)	и двата варианта
Нужна информация	няма	дължина на CPU burst (прогноза)	квант	приоритет
Средно време за чакане	високо	минимално	високо при равни burst-ове	зависи от приоритетите
Време за отговор	лошо	добро за кратките, лошо за дългите	най-добро и предвидимо	добро за важните процеси
Гладуване	не	да (дългите процеси)	не	да, без стареене
Разходи за превключване	минимални	малки / средни	високи при малък квант	малки / средни
Сложност на реализацията	O(1)	O(log n) с пирамида	O(1)	O(log n) или O(1) с опашка за всеки приоритет
Типично приложение	пакетни системи, процеси с еднакъв приоритет	пакетни системи с известни времена, еталон за сравнение	системи с разделяне на времето, интерактивни системи	системи за реално време, основа на съвременните планировчици

Нито един от класическите алгоритми не е най-добър по всички критерии. SJF/SRTF е оптимален по средно време, но изисква информация, която планировчикът няма, и допуска гладуване. RR е справедлив и бърз в отговора, но плаща с превключвания и по-дълго време за обработка. Приоритетното планиране отчита важността, но е податливо на гладуване и инверсия. Затова реалните операционни системи комбинират няколко идеи, както е показано в раздел 7.

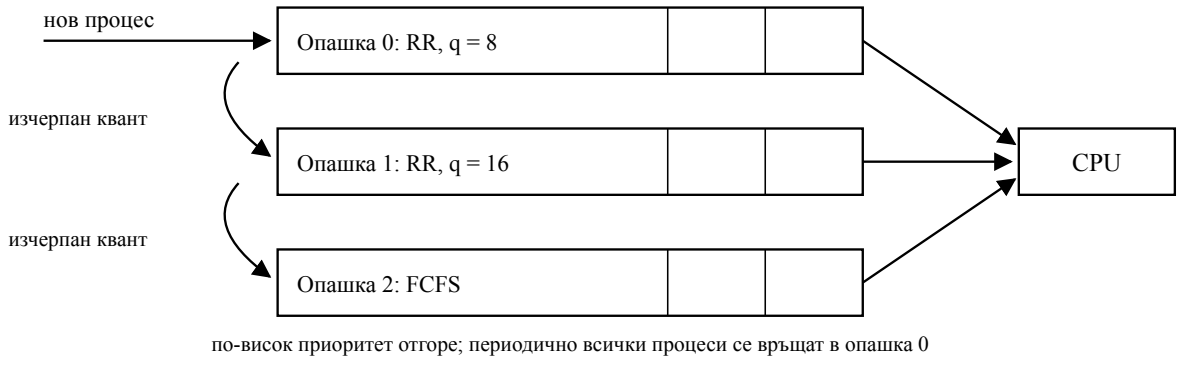
7. ПЛАНИРАНЕ В СЪВРЕМЕННИТЕ ОПЕРАЦИОННИ СИСТЕМИ

7.1. Многостепенни опашки с обратна връзка

При **многостепенните опашки** (multilevel queue) готовите процеси се разделят на няколко опашки, например за системни, интерактивни и фонови процеси. Всяка опашка има собствен алгоритъм (например RR за интерактивните и FCFS за фоновите), а между опашките се прилага приоритетно планиране с изместване или разпределяне на процесорното време.

Многостепенните опашки с обратна връзка (MLFQ, multilevel feedback queue) позволяват на процесите да се преместват между опашките според поведението си. За първи път са използвани от Корбато в системата CTSS в MIT през 1962 г. [9], а идеята им лежи в основата на планировчиците на почти всички по-късни операционни системи. Правилата на MLFQ са [3]:

1. процес с по-висок приоритет винаги се изпълнява преди процес с по-нисък;
2. процеси с еднакъв приоритет се изпълняват по Round Robin;
3. новият процес влиза в опашката с най-висок приоритет;
4. ако процесът изчерпи кванта на своето ниво, той се премества една опашка надолу; по-ниските опашки имат по-голям квант;
5. периодично всички процеси се връщат в най-горната опашка (priority boost), за да не гладуват.



Фиг. 11. Многостепенни опашки с обратна връзка с три нива

MLFQ съчетава предимствата на разгледаните алгоритми без да изисква предварително знание за процесите. Кратките и интерактивните процеси завършват или се блокират още в горните опашки и получават бърз отговор – поведение, близко до SJF. Дългите процеси постепенно слизат надолу и се изпълняват с голям квант и малко превключвания. Връщането в горната опашка играе ролята на стареенето и предотвратява гладуването.

7.2. Linux

Планировчикът на Linux е модулен: всеки процес принадлежи на **клас на планиране** (scheduling class), а класовете се проверяват в строг ред на приоритет. Политиките, достъпни за потребителските програми чрез системното извикване `sched_setscheduler()`, са показани в табл. 13 [7, 8].

Таблица 13. Политики за планиране в Linux

Политика	**Клас**	**Описание**
`SCHED_DEADLINE`	deadline (най-висок)	Планиране по най-ранен краен срок (EDF) с резервиране на процесорно време; от ядро 3.14.
`SCHED_FIFO`	реално време	Статичен приоритет 1–99; в рамките на едно ниво – FCFS без квант. Процесът се изпълнява, докато не се блокира, не завърши или не бъде изместен от процес с по-висок приоритет.
`SCHED_RR`	реално време	Като `SCHED_FIFO`, но в рамките на едно ниво – Round Robin с квант (по подразбиране 100 ms).
`SCHED_OTHER` (`SCHED_NORMAL`), `SCHED_BATCH`, `SCHED_IDLE`	fair (обикновени процеси)	Пропорционално разпределение на процесорното време според стойността *nice* (от −20 до 19); `SCHED_BATCH` е за неинтерактивни задачи, `SCHED_IDLE` – за процеси с най-нисък приоритет.

Класът за реално време е класическо **приоритетно планиране с изместване**, комбинирано с FCFS или RR за процесите с еднакъв приоритет. За да не може грешен процес за реално време да блокира системата, ядрото по подразбиране им позволява да използват най-много 950 ms от всяка секунда (параметрите `sched_rt_runtime_us` и `sched_rt_period_us`).

Обикновените процеси се планират от класа fair. От ядро 2.6.23 (2007 г.) до 6.5 това е **CFS** (Completely Fair Scheduler) [5, 7]. CFS моделира идеален многозадачен процесор, на който всеки от n процеса получава по $1/n$ от мощността едновременно. За всеки процес се пази **виртуално време на изпълнение**

(`vruntime`) – реалното му процесорно време, умножено по коефициент според теглото му. Теглото зависи от *nice*: при nice 0 то е 1024, а всяка стъпка на nice го променя приблизително 1,25 пъти, така че разлика от 1 в nice означава около 10 % разлика в процесорното време. Готовите процеси се пазят в червено-черно дърво, подредено по `vruntime`, и винаги се избира процесът с най-малко `vruntime` – този, който досега е получил най-малко процесорно време. I/O-bound процесите, които често се блокират, натрупват малко `vruntime` и при събуждане бързо получават процесора, т.е. CFS естествено дава предимство на интерактивните процеси, подобно на SJF, но без нужда от прогноза. Фиксиран квант няма: дължината на изпълнението се изчислява динамично от броя на готовите процеси и техните тегла.

От ядро 6.6 (2023 г.) CFS е заменен от **EEVDF** (Earliest Eligible Virtual Deadline First) [8], основан на алгоритъм на Stoica и Abdel-Wahab от 1995 г. [10]. EEVDF запазва идеята за справедливо разпределение по тегла, но добавя на всеки процес **виртуален краен срок**: от процесите, които не са получили повече от полагащото им се (eligible), се избира този с най-ранен краен срок. Процесите с по-кратки заявки за процесорно време получават по-ранни крайни срокове и по-бърз отговор, без това да увеличава общия им дял. Така EEVDF отново съчетава справедливостта на Round Robin с предимството за кратките заявки, характерно за SJF. От ядро 6.12 съществува и механизмът **sched_ext**, който позволява собствени алгоритми за планиране да се зареждат като BPF програми, без промяна на ядрото.

7.3. Windows

Windows използва **приоритетно планиране с изместване** на ниво нишка с 32 нива на приоритет [6, 11]:

- 16–31 – нива за реално време (статични приоритети);
- 1–15 – променливи (динамични) приоритети за обикновените нишки;
- 0 – запазен за системната нишка за нулиране на страници от паметта.

Базовият приоритет на нишката се получава от **класа на приоритет** на процеса (Idle, Below Normal, Normal, Above Normal, High, Realtime) и относителния приоритет на нишката в него. Планировчикът винаги изпълнява готовата нишка с най-висок приоритет, а нишките с еднакъв приоритет се редуват по **Round Robin** с квант. Квантът в клиентските версии е 2 такта на системния таймер (около 30 ms

при такт 15,6 ms), а в сървърните – 12 такта, за да се намалят превключванията. За нишките на активния прозорец квантът може да бъде увеличен.

Динамичните приоритети се коригират с **временно повишаване** (priority boost): нишка получава по-висок приоритет при завършване на входно-изходна операция (например повече при вход от клавиатурата, отколкото от диска), при събуждане след чакане на събитие или когато принадлежи на активния прозорец. След всеки изчерпан квант повишението намалява с едно ниво, докато приоритетът се върне към базовия. Това е идеята на многостепенните опашки с обратна връзка: интерактивните, I/O-bound нишки получават бърз отговор, а CPU-bound нишките се връщат към базовия си приоритет. Срещу гладуването Windows прилага форма на **стареене**: специална системна нишка периодично намира готови нишки, които не са получавали процесора около 4 секунди, и временно повишава приоритета им до 15.

Както при Linux, класическите алгоритми присъстват в Windows като съставни части: приоритетно планиране с изместване, RR за нишки с еднакъв приоритет, обратна връзка по поведението и стареене. Подобен е и планировчикът на macOS (ядро XNU), който също използва многостепенни опашки с приоритети и с понижаване на приоритета на CPU-bound нишките.

ЗАКЛЮЧЕНИЕ

В курсовата работа бяха разгледани четирите класически алгоритъма за планиране на процесорното време – FCFS, SJF/SRTF, Round Robin и приоритетното планиране – и беше показано, че всеки от тях оптимизира различен критерий за сметка на други.

Основните изводи са:

- **FCFS** е най-простият и не допуска гладуване, но заради ефекта на конвоя има най-високо средно време за чакане и лош отговор. В симулацията с 200 процеса средното му време за чакане е около пет пъти по-голямо от това при SRTF.
- **SJF и SRTF** са доказуемо оптимални по средно време за чакане и служат за еталон, но изискват прогнозиране на CPU burst и водят до гладуване на дългите процеси – в симулацията максималното време за чакане е около три пъти по-голямо от това при FCFS.
- **Round Robin** осигурява справедливо разпределение и най-доброто време за отговор, затова е основа на системите с разделяне на времето. Ефективността му зависи критично от кванта: при твърде малък квант почти половината процесорно време се губи за превключване, а при твърде голям RR се превръща във FCFS.
- **Приоритетното планиране** отчита важността на процесите и е задължително за системите за реално време, но без стареене води до гладуване, а при споделени ресурси – до инверсия на приоритетите. Стареенето намали максималното време за чакане в симулацията повече от два пъти.
- Съвременните операционни системи не използват нито един от алгоритмите в чист вид, а ги комбинират: Windows – приоритетно планиране с RR, временно повишаване на приоритетите и стареене; Linux – приоритетно планиране за процесите за реално време и справедливо разпределение (CFS, а от ядро 6.6 – EEVDF) за обикновените процеси.

Поставената цел е изпълнена: алгоритмите са описани, приложени върху общ пример и сравнени количествено с помощта на симулатор, чийто код е приложен и позволява изследването да бъде повторено с други входни данни. Възможни

направления за по-нататъшна работа са планирането в многопроцесорни системи (балансиране на натоварването, процесорна привързаност), алгоритмите за реално време (Rate Monotonic, EDF) и експериментално сравнение на CFS и EEVDF върху реално ядро на Linux.

ИЗПОЛЗВАНА ЛИТЕРАТУРА

1. Silberschatz, A., Galvin, P. B., Gagne, G. *Operating System Concepts*, 10th ed. Hoboken: Wiley, 2018. Chapter 5: CPU Scheduling.
2. Tanenbaum, A. S., Bos, H. *Modern Operating Systems*, 4th ed. Boston: Pearson, 2014. Section 2.4: Scheduling.
3. Arpaci-Dusseau, R. H., Arpaci-Dusseau, A. C. *Operating Systems: Three Easy Pieces*, version 1.10. Arpaci-Dusseau Books, 2023. Chapters 7–9. https://pages.cs.wisc.edu/~remzi/OSTEP/ (достъп: 13.10.2026).
4. Stallings, W. *Operating Systems: Internals and Design Principles*, 9th ed. Boston: Pearson, 2018. Chapter 9: Uniprocessor Scheduling.
5. Love, R. *Linux Kernel Development*, 3rd ed. Upper Saddle River: Addison-Wesley, 2010. Chapter 4: Process Scheduling.
6. Yosifovich, P., Ionescu, A., Russinovich, M. E., Solomon, D. A. *Windows Internals, Part 1*, 7th ed. Redmond: Microsoft Press, 2017. Chapter 4: Threads.
7. The Linux Kernel Documentation. *CFS Scheduler*. https://docs.kernel.org/scheduler/sched-design-CFS.html (достъп: 13.10.2026).
8. The Linux Kernel Documentation. *EEVDF Scheduler*. https://docs.kernel.org/scheduler/sched-eevdf.html; Linux man-pages, *sched(7)*. https://man7.org/linux/man-pages/man7/sched.7.html (достъп: 13.10.2026).
9. Corbató, F. J., Merwin-Daggett, M., Daley, R. C. An Experimental Time-Sharing System. In: *Proceedings of the AFIPS Spring Joint Computer Conference*, 1962, pp. 335–344.
10. Stoica, I., Abdel-Wahab, H. *Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation*. Technical Report TR-95-22, Old Dominion University, 1995.
11. Microsoft Learn. *Scheduling Priorities*; *Priority Boosts*. https://learn.microsoft.com/en-us/windows/win32/procthread/scheduling-priorities (достъп: 13.10.2026).

ПРИЛОЖЕНИЕ А. ПРОГРАМА ЗА СИМУЛИРАНЕ НА АЛГОРИТМИТЕ

Програмата `sim.py` е написана на Python 3 и използва само стандартната библиотека. Изпълнена без аргументи, тя отпечатва всички примери и експерименти от реферата. С аргумент – име на CSV файл с редове „име, пристигане, burst, приоритет“ – симулира собствен набор от процеси; квантът за Round Robin се задава с `-q`:

```
python3 sim.py
python3 sim.py procesi.csv -q 4
```

```
#!/usr/bin/env python3
"""CPU scheduling simulator: FCFS, SJF, SRTF, Round Robin, priority (non-preemptive,
preemptive and non-preemptive with aging).

Time is discrete (one unit per step). For every algorithm the simulator returns the
Gantt chart and, per process, completion, turnaround, waiting and response time.

  python3 sim.py                      # examples and experiments used in the реферат
  python3 sim.py procs.csv [-q 4]     # own workload: lines
"name,arrival,burst,priority"

Conventions:
  * smaller priority number = higher priority;
  * ties are broken by arrival time, then by input order;
  * in Round Robin a process that arrives at the same moment its predecessor's
    quantum expires joins the queue before the preempted process;
  * SRTF and preemptive priority preempt only on a strictly better candidate;
  * aging raises a waiting process by one priority level per AGING_STEP units in the
ready queue.
"""
import random
import sys
from collections import deque
from dataclasses import dataclass, field

ALGORITHMS = {
    "fcfs": "FCFS",
    "sjf": "SJF",
    "srtf": "SRTF",
    "rr": "Round Robin",
    "prio": "Приоритетно (без изместване)",
    "prio-p": "Приоритетно (с изместване)",
    "prio-a": "Приоритетно със стареене",
}
AGING_STEP = 50


@dataclass
class Proc:
    name: str
    arrival: int
    burst: int
    priority: int = 0
```

```
    index: int = 0


@dataclass
class Result:
    gantt: list = field(default_factory=list)
    completion: dict = field(default_factory=dict)
    first_run: dict = field(default_factory=dict)
    switches: int = 0
    procs: list = field(default_factory=list)

    def turnaround(self, p):
        return self.completion[p.name] - p.arrival

    def waiting(self, p):
        return self.turnaround(p) - p.burst

    def response(self, p):
        return self.first_run[p.name] - p.arrival

    def avg(self, metric):
        return sum(getattr(self, metric)(p) for p in self.procs) / len(self.procs)

    def worst(self, metric):
        return max(getattr(self, metric)(p) for p in self.procs)

    @property
    def length(self):
        return self.gantt[-1][2] if self.gantt else 0

    @property
    def cs_time(self):
        return sum(e - s for n, s, e in self.gantt if n == "CS")


def simulate(procs, alg, quantum=None, cs_cost=0):
    if alg not in ALGORITHMS:
        raise ValueError(f"unknown algorithm: {alg}")
    if alg == "rr" and not quantum:
        raise ValueError("Round Robin needs a quantum")
    procs = [Proc(p.name, p.arrival, p.burst, p.priority, i) for i, p in
enumerate(procs)]
    pending = deque(sorted(procs, key=lambda p: (p.arrival, p.index)))
    remaining = {p.name: p.burst for p in procs}
    ready = deque()
    since = {}
    res = Result(procs=procs)
    current, last, used, cs_left, t = None, None, 0, 0, 0

    def key(p):
        if alg in ("sjf", "srtf"):
            return (remaining[p.name], p.arrival, p.index)
        if alg in ("prio", "prio-p"):
            return (p.priority, p.arrival, p.index)
        if alg == "prio-a":
            return (p.priority - (t - since[p.name]) // AGING_STEP, p.arrival, p.index)
        return (p.arrival, p.index)

    def record(name):
        if res.gantt and res.gantt[-1][0] == name and res.gantt[-1][2] == t:
            res.gantt[-1][2] = t + 1
        else:
```

```
            res.gantt.append([name, t, t + 1])

    while pending or ready or current:
        while pending and pending[0].arrival <= t:
            since[pending[0].name] = t
            ready.append(pending.popleft())

        if current and cs_left == 0:
            preempt = False
            if alg == "rr":
                preempt = used >= quantum
            elif alg == "srtf" and ready:
                preempt = key(min(ready, key=key)) < key(current)
            elif alg == "prio-p" and ready:
                preempt = min(p.priority for p in ready) < current.priority
            if preempt:
                since[current.name] = t
                ready.append(current)
                current = None

        if current is None and ready:
            if alg == "rr":
                current = ready.popleft()
            else:
                current = min(ready, key=key)
                ready.remove(current)
            used = 0
            if last is not None and current is not last:
                res.switches += 1
                cs_left = cs_cost

        if current is None:
            record(None)
        elif cs_left:
            record("CS")
            cs_left -= 1
        else:
            record(current.name)
            res.first_run.setdefault(current.name, t)
            remaining[current.name] -= 1
            used += 1
            last = current
            if remaining[current.name] == 0:
                res.completion[current.name] = t + 1
                current = None
        t += 1

    res.gantt = [tuple(g) for g in res.gantt]
    return res


EXAMPLE = [
    Proc("P1", 0, 8, 3),
    Proc("P2", 1, 4, 1),
    Proc("P3", 2, 9, 4),
    Proc("P4", 3, 5, 2),
    Proc("P5", 4, 2, 5),
]

CONVOY_LONG_FIRST = [Proc("P1", 0, 24), Proc("P2", 0, 3), Proc("P3", 0, 3)]
CONVOY_SHORT_FIRST = [Proc("P2", 0, 3), Proc("P3", 0, 3), Proc("P1", 0, 24)]
```

```
EXAMPLE_QUANTUM = 3
QUANTA = [1, 2, 4, 8, 16, 32, 64, 128, 256]
CS_COST = 1


def workload(n=200, seed=2026):
    """Mixed load: 80 % short interactive bursts (2-20), 20 % long CPU-bound bursts (50-
200)."""
    rnd = random.Random(seed)
    procs, t = [], 0
    for i in range(n):
        t += int(rnd.expovariate(1 / 38))
        burst = rnd.randint(2, 20) if rnd.random() < 0.8 else rnd.randint(50, 200)
        procs.append(Proc(f"P{i + 1}", t, burst, rnd.randint(1, 8)))
    return procs


def runs(procs, quantum):
    order = [("fcfs", None), ("sjf", None), ("srtf", None), ("rr", quantum), ("prio",
None), ("prio-p", None),
             ("prio-a", None)]
    return [(alg, q, simulate(procs, alg, q)) for alg, q in order]


def label(alg, q):
    return f"{ALGORITHMS[alg]} (q = {q})" if q else ALGORITHMS[alg]


def gantt_text(res):
    return " ".join(f"[{s}-{e} {n or 'idle'}]" for n, s, e in res.gantt)


def print_table(procs, res):
    print(f"  {'Процес':<7}{'Пристигане':>11}{'Burst':>7}{'Приор.':>7}{'Край':>6}
{'Обраб.':>8}{'Чакане':>8}{'Отговор':>9}")
    for p in res.procs:
        print(f"  {p.name:<7}{p.arrival:>11}{p.burst:>7}{p.priority:>7}
{res.completion[p.name]:>6}"
              f"{res.turnaround(p):>8}{res.waiting(p):>8}{res.response(p):>9}")
    print(f"  средно: обработка {res.avg('turnaround'):.2f}, чакане
{res.avg('waiting'):.2f}, "
          f"отговор {res.avg('response'):.2f}, превключвания {res.switches}")


def print_summary(procs, quantum):
    print(f"  {'Алгоритъм':<34}{'ср. обраб.':>11}{'ср. чакане':>11}{'ср. отговор':>12}
{'макс. чакане':>13}{'превкл.':>9}")
    for alg, q, res in runs(procs, quantum):
        print(f"  {label(alg, q):<34}{res.avg('turnaround'):>11.2f}
{res.avg('waiting'):>11.2f}"
              f"{res.avg('response'):>12.2f}{res.worst('waiting'):>13}
{res.switches:>9}")


def quantum_sweep(procs):
    rows = []
    for q in QUANTA:
        res = simulate(procs, "rr", q, CS_COST)
        rows.append((q, res.avg("turnaround"), res.avg("response"), res.switches, 100 *
res.cs_time / res.length))
    return rows
```

```
def main(argv):
    if len(argv) > 1 and argv[1] not in ("-q",):
        quantum = int(argv[argv.index("-q") + 1]) if "-q" in argv else EXAMPLE_QUANTUM
        procs = []
        with open(argv[1], encoding="utf-8") as f:
            for line in f:
                if line.strip() and not line.startswith("#"):
                    name, arrival, burst, *prio = [x.strip() for x in line.split(",")]
                    procs.append(Proc(name, int(arrival), int(burst), int(prio[0]) if
prio else 0))
        for alg, q, res in runs(procs, quantum):
            print(f"\n{label(alg, q)}\n  {gantt_text(res)}")
            print_table(procs, res)
        return

    print("Ефект на конвоя (FCFS):")
    for procs in (CONVOY_LONG_FIRST, CONVOY_SHORT_FIRST):
        res = simulate(procs, "fcfs")
        print(f"  {gantt_text(res)}  средно чакане {res.avg('waiting'):.2f}")

    print(f"\nОбщ пример, q = {EXAMPLE_QUANTUM}:")
    for alg, q, res in runs(EXAMPLE, EXAMPLE_QUANTUM):
        print(f"\n{label(alg, q)}\n  {gantt_text(res)}")
        print_table(EXAMPLE, res)

    load = workload()
    total = sum(p.burst for p in load)
    print(f"\nСмесено натоварване: {len(load)} процеса, общо {total} единици CPU време,
"
          f"последно пристигане {load[-1].arrival}")
    for q in (4, 20):
        print(f"\nq = {q}:")
        print_summary(load, q)

    print(f"\nRound Robin при различен квант, цена на превключването {CS_COST}:")
    print(f"  {'q':>5}{'ср. обраб.':>12}{'ср. отговор':>13}{'превкл.':>9}{'загуба
%':>10}")
    for q, tat, resp, sw, loss in quantum_sweep(load):
        print(f"  {q:>5}{tat:>12.2f}{resp:>13.2f}{sw:>9}{loss:>10.2f}")


if __name__ == "__main__":
    main(sys.argv)
```